# WIRESHARK PACKET ANALYSIS

## 1.Introduction

Wireshark is a network protocol analyzer used to capture and examine network traffic in real time.Wireshark is extremely helpful in helping users understand how packets actually move through the network in real time-It's especially helpful for understanding commonly used networking protocols like-TCP,DNS or HTTP.By analyzing these packets users can see how connections are established(TCP three way handshake),How DNS queries are resolved etc. Alongside Wireshark, Nmap was used for host discovery and port scanning to provide a broader view of the network environment during the analysis session

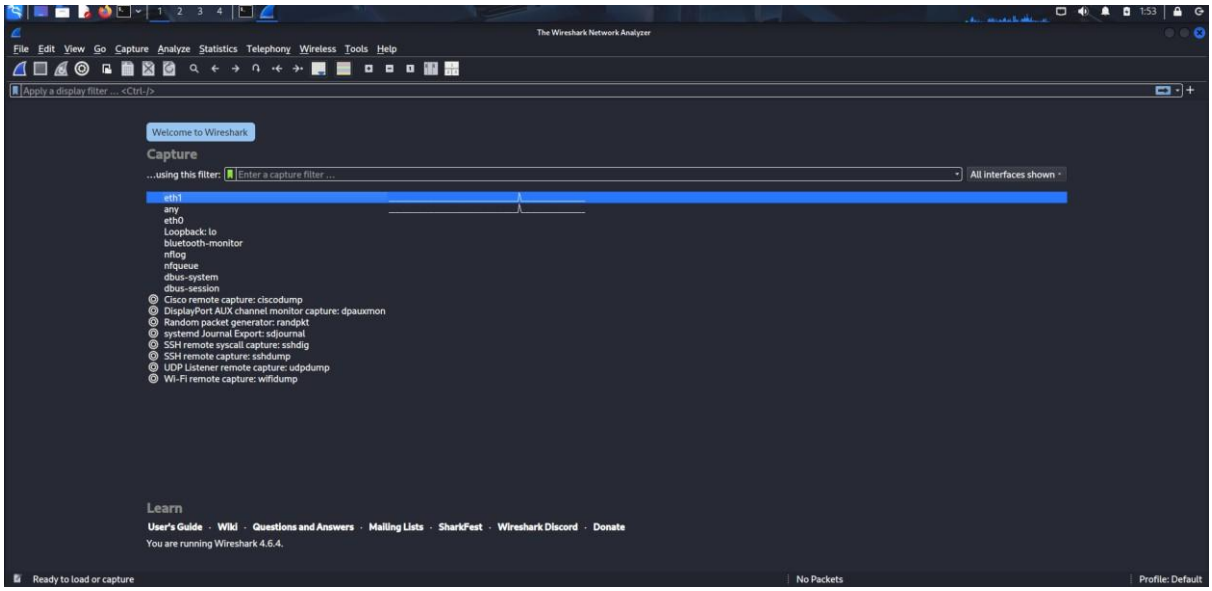



*Figure 1:Wireshark home screen*

This is the wireshark home screen showing all the available interfaces. From this window we can select through which interface we want to monitor network traffic.

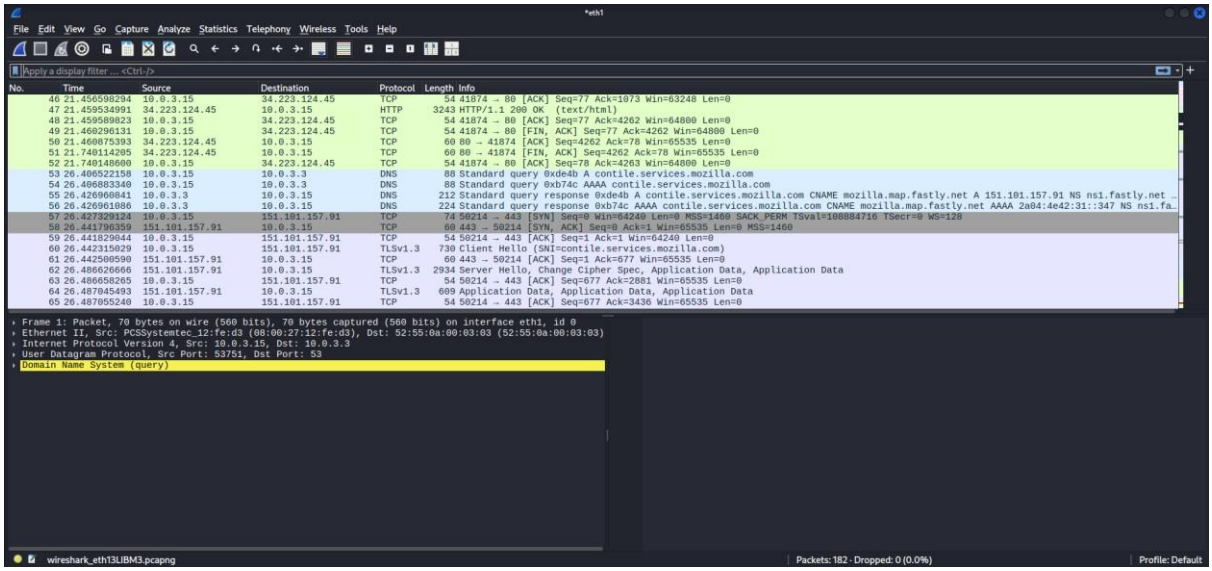


*Figure 2:Captured packets*

Figure 2 shows the packet capture screen in Wireshark after a few minutes of network activity, including browsing, pinging websites etc. This list displays captures with source IP address, destination IP address etc and for a more in depth view we can left click on individual packets and view a detailed breakdown on the bottom left pane.

# 2.Packet Analysis

## 2.1 DNS

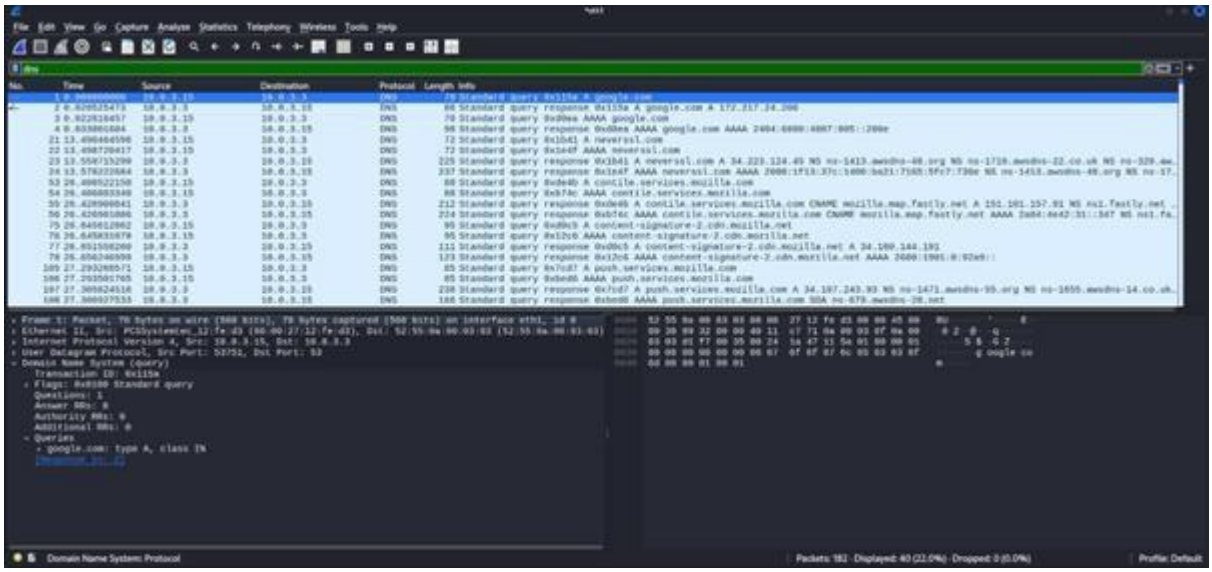


*Figure 3:DNS Packet Filtered View*

Figure 3 shows a filtered view of only DNS packets by using the DNS filter. By applying this filter, only packets related to the **Domain Name System (DNS)** are displayed, allowing easier analysis of DNS traffic. The packet list shows multiple DNS queries and responses generated when accessing websites. Each entry contains details such as the source and destination IP addresses, packet length, and query information.

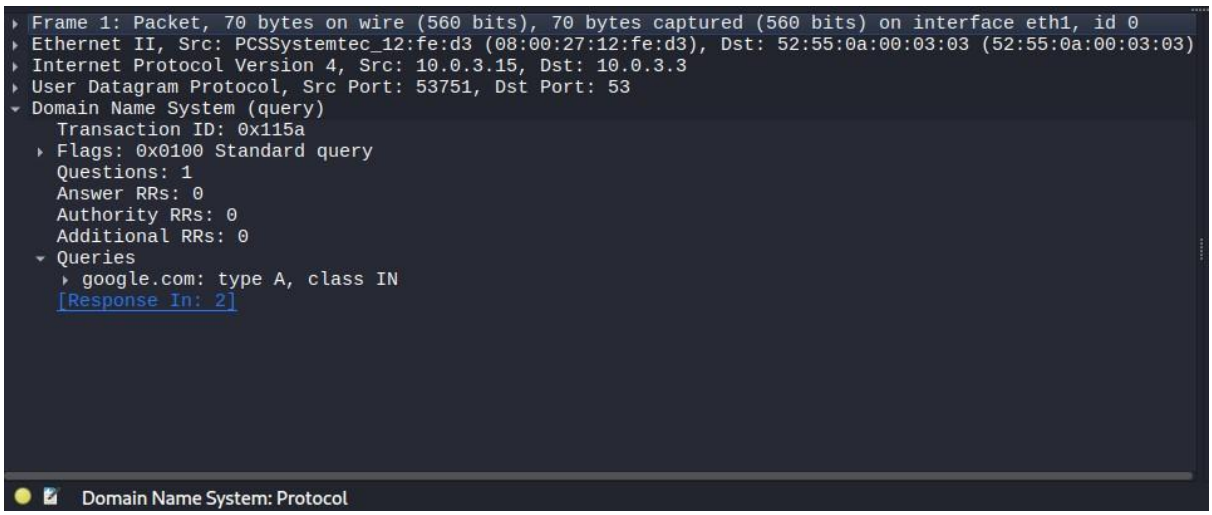



*Figure 4:Detailed DNS Packet Inspection*

Figure 4 shows the detailed analysis of a selected DNS packet. The packet details pane displays the different protocol layers involved in the communication, including Ethernet, Internet Protocol (IP), User Datagram Protocol (UDP), and DNS. The DNS section shows information such as the transaction ID, query type, flags, and the domain name being queried (google.com).

DNS traffic runs over UDP port 53, meaning these queries are transmitted in plaintext with no encryption by default. Any observer on the network can see exactly which domains are being resolved

## 2.2 ICMP

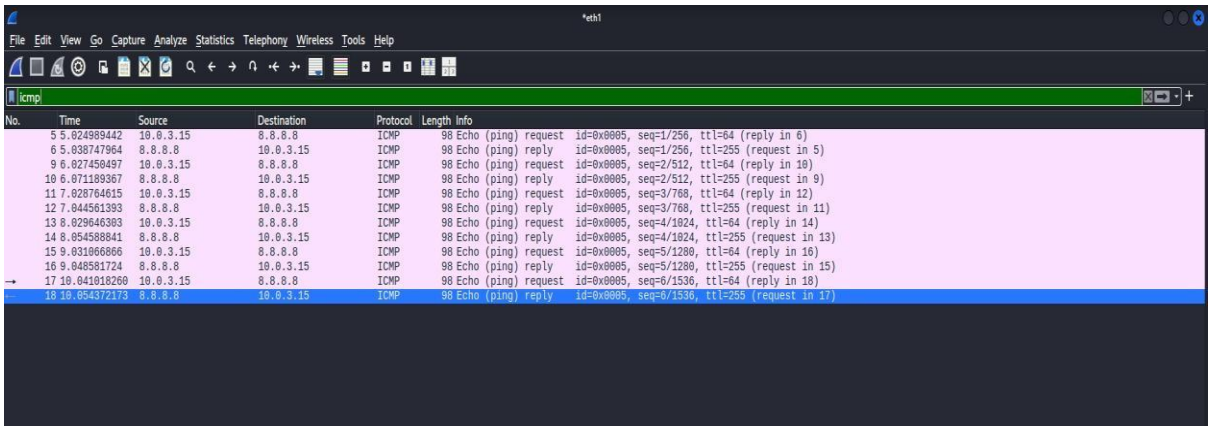


*Figure 5:ICMP Ping Packet Capture*

Figure 5 shows ICMP packets captured in Wireshark after applying the ICMP filter. The packets represent ping requests and replies exchanged between the local system and the destination IP address 8.8.8.8 (Google DNS server). The packet list displays details such as the source IP address, destination IP address, protocol type (ICMP), packet length, sequence numbers, and Time To Live (TTL) values. These packets demonstrate how the ping command tests network connectivity by sending ICMP Echo Requests and receiving Echo Replies, confirming that communication between the devices is successful.

**<u>NOTE: The source IP address is 10.0.3.15</u>**

## 2.3 HTTP

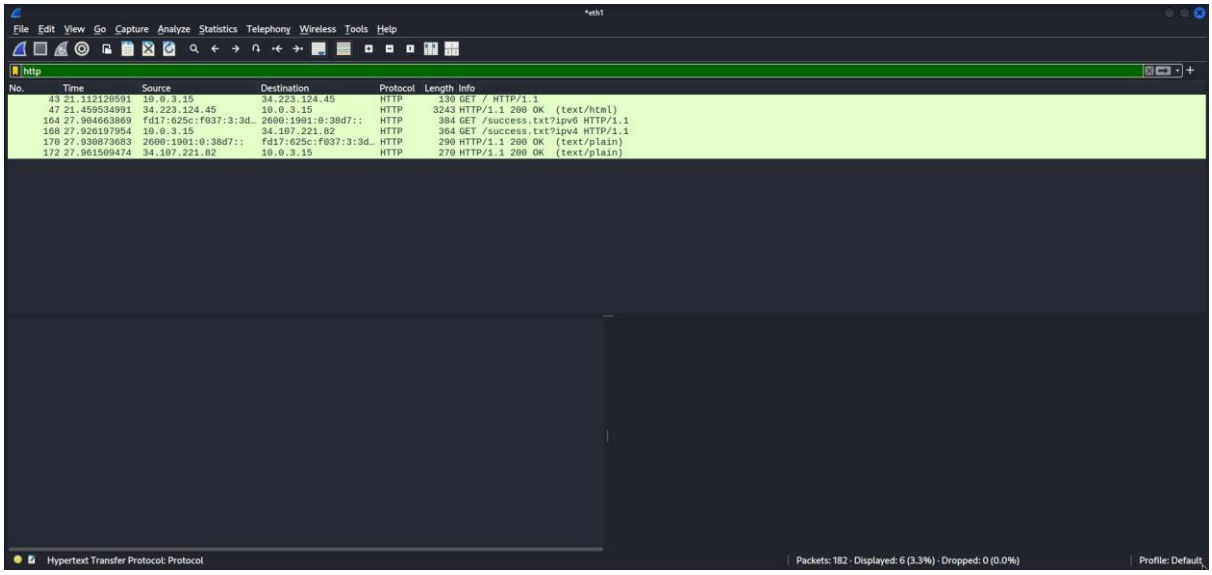



*Figure 6:Filtered HTTP Packets*

Figure 6 shows HTTP packets after applying the HTTP filter in Wireshark. The captured packets include HTTP requests and responses generated while accessing a website. The packet list shows communication between the local system (10.0.3.15) and the remote server (34.223.124.45) using the HTTP protocol over port 80. Entries such as GET requests and OK responses indicate that the client requested web resources and the server successfully returned the requested content.

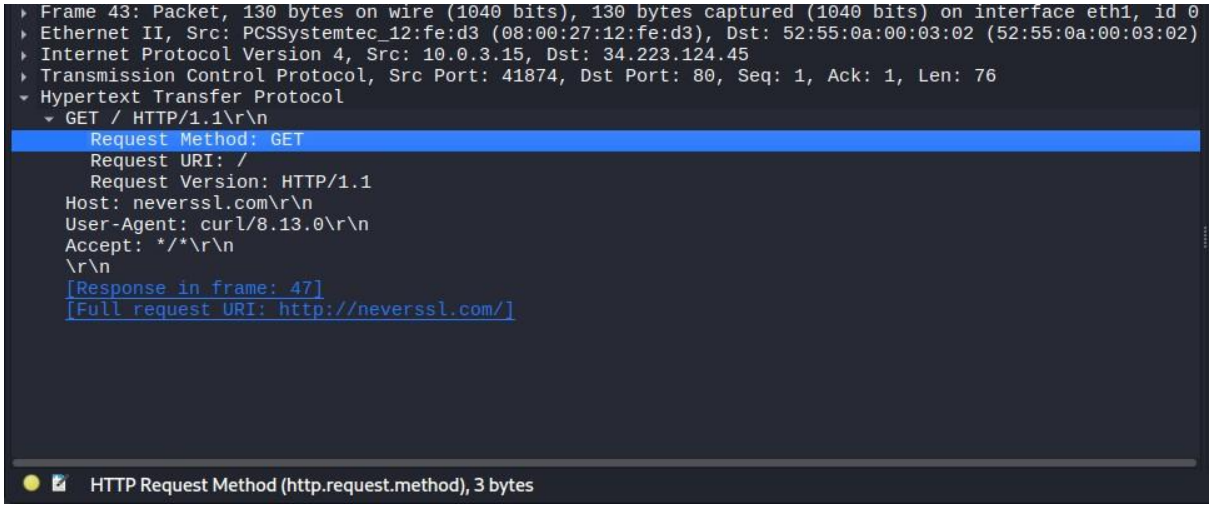



*Figure 7:Detailed HTTP Request Packet Inspection*

Figure 7 displays the detailed breakdown of a selected HTTP packet. The HTTP section reveals important information such as the request method (GET), request URI, host name (neverssl.com), HTTP version, it also specifies exactly which frame the response for this GET request is provided (frame 47). This helps in understanding how a client sends a request to retrieve a web page from a server.

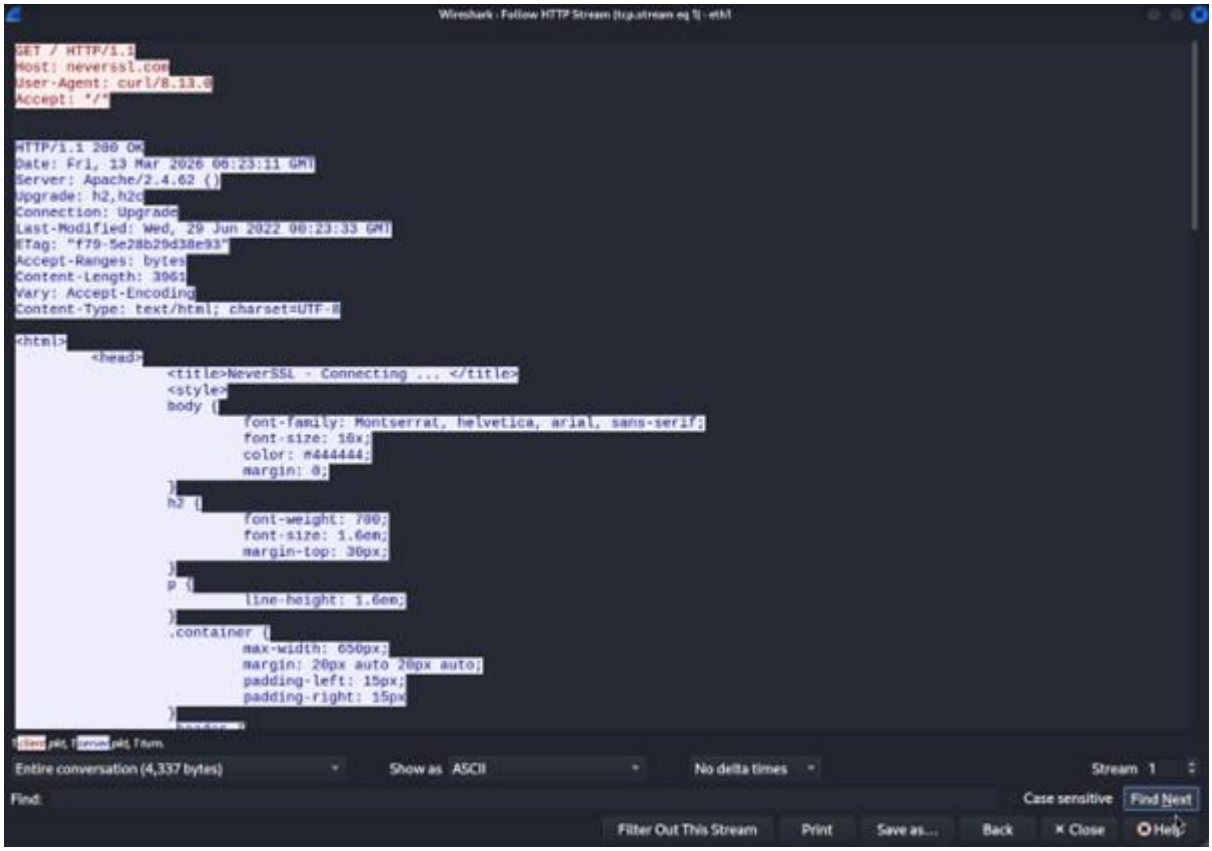
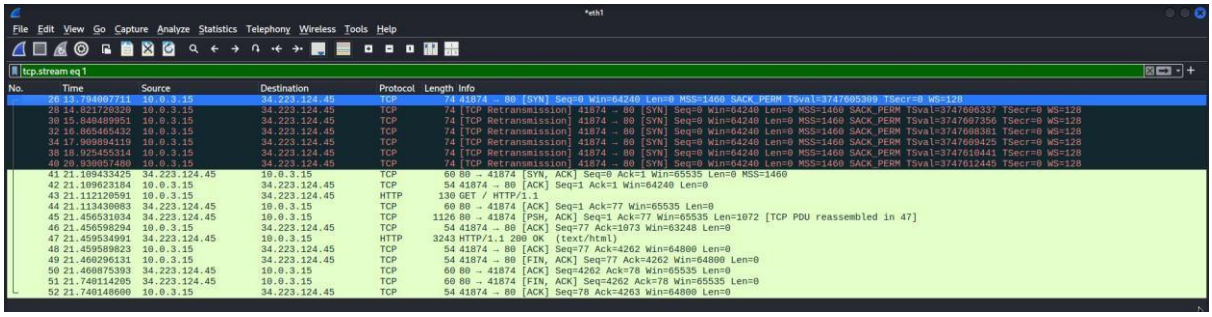



*Figure 8:Follow TCP Stream Output*

If we right click on a specific packet and hover over follow on click on follow TCP stream we get this output. This is a reconstruction of the communication between server and client in readable form. Figure 8 shows the GET request and the OK response followed by the HTML content of the webpage.

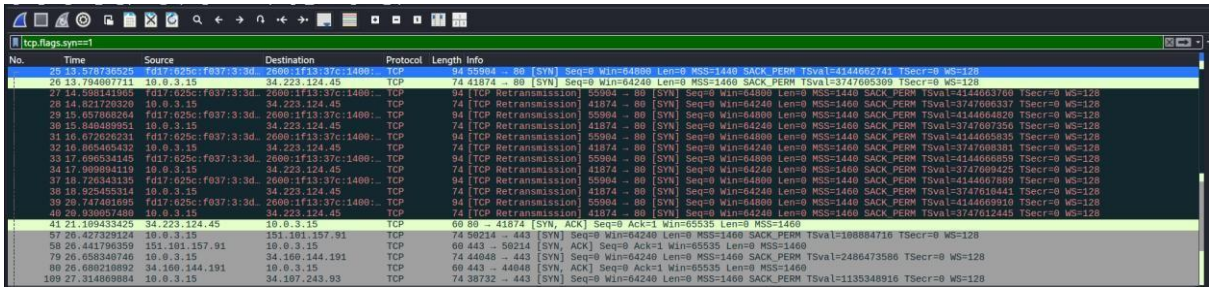
## 2.4 TCP



| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 26 | 13.794007711 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747605309 TSecr=0 WS=128 |
| 28 | 14.821720320 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747606337 TSecr=0 WS=128 |
| 30 | 15.840489951 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747607356 TSecr=0 WS=128 |
| 32 | 16.865465432 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747608381 TSecr=0 WS=128 |
| 34 | 17.909894119 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747609425 TSecr=0 WS=128 |
| 38 | 18.925455314 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747610441 TSecr=0 WS=128 |
| 40 | 20.930057480 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747612445 TSecr=0 WS=128 |
| 41 | 21.109433425 | 34.223.124.45 | 10.0.3.15 | TCP | 60 | 80 → 41874 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 |
| 42 | 21.109623184 | 10.0.3.15 | 34.223.124.45 | TCP | 54 | 41874 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0 |
| 43 | 21.112120591 | 10.0.3.15 | 34.223.124.45 | HTTP | 130 | GET / HTTP/1.1 |
| 44 | 21.113430083 | 34.223.124.45 | 10.0.3.15 | TCP | 60 | 80 → 41874 [ACK] Seq=1 Ack=77 Win=65535 Len=0 |
| 45 | 21.456531034 | 34.223.124.45 | 10.0.3.15 | TCP | 1126 | 80 → 41874 [PSH, ACK] Seq=1 Ack=77 Win=65535 Len=1072 [TCP PDU reassembled in 47] |
| 46 | 21.456598294 | 10.0.3.15 | 34.223.124.45 | TCP | 54 | 41874 → 80 [ACK] Seq=77 Ack=1073 Win=63248 Len=0 |
| 47 | 21.459534991 | 34.223.124.45 | 10.0.3.15 | HTTP | 3243 | HTTP/1.1 200 OK (text/html) |
| 48 | 21.459589823 | 10.0.3.15 | 34.223.124.45 | TCP | 54 | 41874 → 80 [ACK] Seq=77 Ack=4262 Win=64800 Len=0 |
| 49 | 21.460296131 | 10.0.3.15 | 34.223.124.45 | TCP | 54 | 41874 → 80 [FIN, ACK] Seq=77 Ack=4262 Win=64800 Len=0 |
| 50 | 21.460875393 | 34.223.124.45 | 10.0.3.15 | TCP | 60 | 80 → 41874 [ACK] Seq=4262 Ack=78 Win=65535 Len=0 |
| 51 | 21.740114205 | 34.223.124.45 | 10.0.3.15 | TCP | 60 | 80 → 41874 [FIN, ACK] Seq=4262 Ack=78 Win=65535 Len=0 |
| 52 | 21.740148600 | 10.0.3.15 | 34.223.124.45 | TCP | 54 | 41874 → 80 [ACK] Seq=78 Ack=4263 Win=64800 Len=0 |

*Figure 9:TCP Stream Packets*

Figure 9 shows the TCP packets for the same HTTP packet we followed the TCP frame for. Here is where we can clearly see how connections are established via the three way handshake. The source(10.0.3.15) sends an initial SYN packet to the destination(34.223.124.45) as the 26$^{th}$ frame. The SYN-ACK is not sent from the destination until the 41$^{st}$ frame so we get to see TCP retransmission in play, the source keeps retransmitting the SYN packet as it is not receiving an acknowledgement. At the 42$^{nd}$ frame the source sends the ACK packet and hence connection is established after which the HTTP GET request is made.



| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 25 | 13.578736525 | fd17:625c:f037:3:3d… | 2600:1f13:37c:1400:… | TCP | 94 | 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144662741 TSecr=0 WS=128 |
| 26 | 13.794007711 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747605309 TSecr=0 WS=128 |
| 27 | 14.598141905 | fd17:625c:f037:3:3d… | 2600:1f13:37c:1400:… | TCP | 94 | [TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144663760 TSecr=0 WS=128 |
| 28 | 14.821720320 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747606337 TSecr=0 WS=128 |
| 29 | 15.657868264 | fd17:625c:f037:3:3d… | 2600:1f13:37c:1400:… | TCP | 94 | [TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144664820 TSecr=0 WS=128 |
| 30 | 15.840489951 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747607356 TSecr=0 WS=128 |
| 31 | 16.672626231 | fd17:625c:f037:3:3d… | 2600:1f13:37c:1400:… | TCP | 94 | [TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144665835 TSecr=0 WS=128 |
| 32 | 16.865465432 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747608381 TSecr=0 WS=128 |
| 33 | 17.696534145 | fd17:625c:f037:3:3d… | 2600:1f13:37c:1400:… | TCP | 94 | [TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144666859 TSecr=0 WS=128 |
| 34 | 17.909894119 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747609425 TSecr=0 WS=128 |
| 37 | 18.726343135 | fd17:625c:f037:3:3d… | 2600:1f13:37c:1400:… | TCP | 94 | [TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144667889 TSecr=0 WS=128 |
| 38 | 18.925455314 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747610441 TSecr=0 WS=128 |
| 39 | 20.747461695 | fd17:625c:f037:3:3d… | 2600:1f13:37c:1400:… | TCP | 94 | [TCP Retransmission] 55904 → 80 [SYN] Seq=0 Win=64800 Len=0 MSS=1440 SACK_PERM TSval=4144669910 TSecr=0 WS=128 |
| 40 | 20.930057480 | 10.0.3.15 | 34.223.124.45 | TCP | 74 | [TCP Retransmission] 41874 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=3747612445 TSecr=0 WS=128 |
| 41 | 21.109433425 | 34.223.124.45 | 10.0.3.15 | TCP | 60 | 80 → 41874 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 |
| 57 | 26.427329124 | 10.0.3.15 | 151.101.157.91 | TCP | 74 | 50214 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=108884716 TSecr=0 WS=128 |
| 58 | 26.441796359 | 151.101.157.91 | 10.0.3.15 | TCP | 60 | 443 → 50214 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 |
| 79 | 26.658340746 | 10.0.3.15 | 34.160.144.191 | TCP | 74 | 44048 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2486473586 TSecr=0 WS=128 |
| 80 | 26.680210892 | 34.160.144.191 | 10.0.3.15 | TCP | 60 | 443 → 44048 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 |
| 109 | 27.314869884 | 10.0.3.15 | 34.107.243.93 | TCP | 74 | 38732 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1135348916 TSecr=0 WS=128 |

*Figure 10:TCP SYN Packets*

Figure 10 shows TCP packets filtered using the tcp.flags.syn==1 filter, which displays packets used to initiate TCP connections. The highlighted packets represent SYN packets sent by the client to start communication with the server

```
    1010 .... = Header Length: 40 bytes (10)
  ▾ Flags: 0x002 (SYN)
      000. .... .... = Reserved: Not set
      ...0 .... .... = Accurate ECN: Not set
      .... 0... .... = Congestion Window Reduced: Not set
      .... .0.. .... = ECN-Echo: Not set
      .... ..0. .... = Urgent: Not set
      .... ...0 .... = Acknowledgment: Not set
      .... .... 0... = Push: Not set
      .... .... .0.. = Reset: Not set
    ▸ .... .... ..1. = Syn: Set
      .... .... ...0 = Fin: Not set
      [TCP Flags: ··********S·]
    Window: 64800
    [Calculated window size: 64800]
    Checksum: 0x1eae [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
  ▸ Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window sc
  ▸ [Timestamps]
    [Client Contiguous Streams: 0]
    [Server Contiguous Streams: 1]
```

*Figure 11:Detailed TCP SYN Flag Inspection*

Figure 11 displays the detailed analysis of a TCP SYN packet in the packet details pane. The TCP header shows that the SYN (Synchronize) flag is set, indicating that the packet is part of the TCP three-way handshake used to establish a connection. Additional details such as the TCP window size, checksum, and TCP options are also visible, which help manage data flow and reliability during communication.

# 3. Nmap Port and Service Discovery Scan

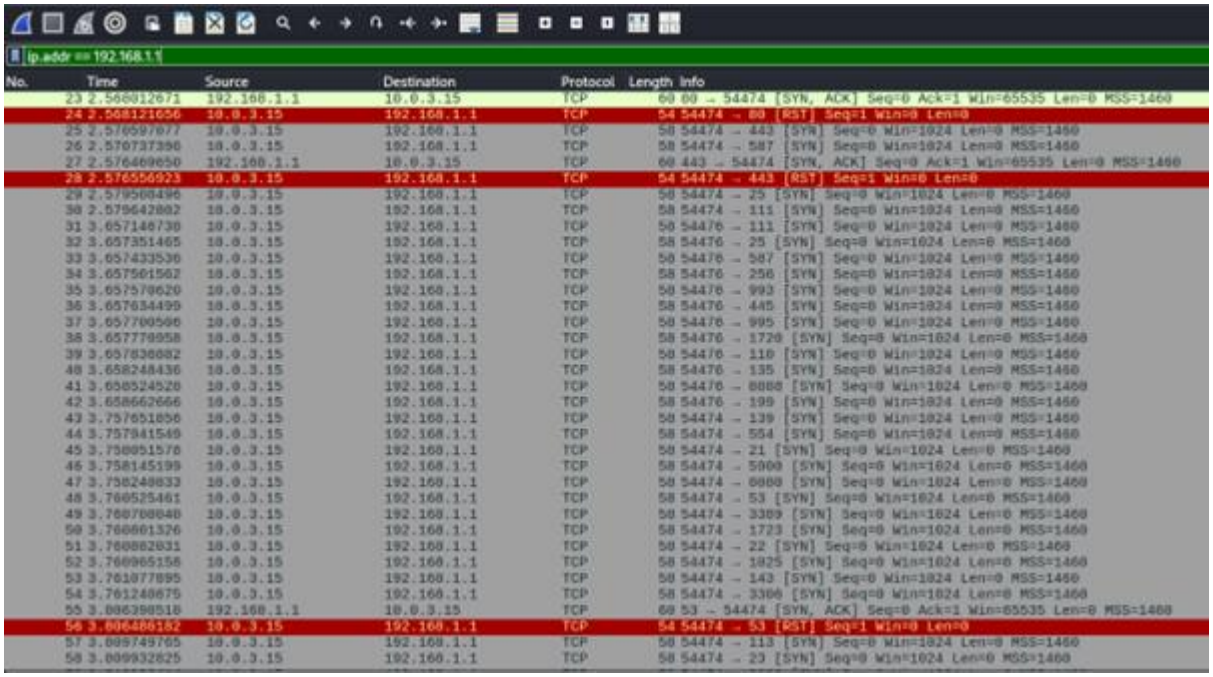
```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 192.168.1.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-13 09:05 -0400
Stats: 0:00:30 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 66.67% done; ETC: 09:05 (0:00:12 remaining)
Stats: 0:01:18 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 66.67% done; ETC: 09:06 (0:00:36 remaining)
Nmap scan report for 192.168.1.1
Host is up (0.0065s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT    STATE SERVICE    VERSION
53/tcp  open  domain     dnsmasq
80/tcp  open  http       mini_httpd x.x
443/tcp open  ssl/https?

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 144.87 seconds

┌──(kali㉿kali)-[~]
└─$
```

*Figure 12:Nmap Service Version Scan Results*

Figure 12 shows the results of an Nmap service version scan (nmap -sV) performed on the target host 192.168.1.1. The scan identifies open ports and attempts to determine the services running on those ports. The results indicate that port 53 (DNS), port 80 (HTTP), and port 443 (HTTPS) are open on the target system. The scan also identifies the services associated with these ports, such as dnsmasq for DNS and mini_httpd for HTTP, providing useful information about the target system's network services and configuration.



ip.addr == 192.168.1.1

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 23 | 2.568012671 | 192.168.1.1 | 10.0.3.15 | TCP | 60 | 80 → 54474 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 |
| 24 | 2.568121656 | 10.0.3.15 | 192.168.1.1 | TCP | 54 | 54474 → 80 [RST] Seq=1 Win=0 Len=0 |
| 25 | 2.576597077 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 443 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 26 | 2.570737390 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 27 | 2.576469650 | 192.168.1.1 | 10.0.3.15 | TCP | 60 | 443 → 54474 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 |
| 28 | 2.576556923 | 10.0.3.15 | 192.168.1.1 | TCP | 54 | 54474 → 443 [RST] Seq=1 Win=0 Len=0 |
| 29 | 2.579560496 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 30 | 2.579642802 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 31 | 3.657148730 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 111 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 32 | 3.657351465 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 25 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 33 | 3.657433530 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 587 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 34 | 3.657561562 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 256 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 35 | 3.657579620 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 993 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 36 | 3.657634499 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 445 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 37 | 3.657700506 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 995 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 38 | 3.657778958 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 1720 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 39 | 3.657836082 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 110 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 40 | 3.658248436 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 135 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 41 | 3.658524520 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 42 | 3.658662666 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54476 → 199 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 43 | 3.757651856 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 139 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 44 | 3.757841549 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 554 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 45 | 3.758051578 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 21 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 46 | 3.758145199 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 5900 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 47 | 3.758248833 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 8080 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 48 | 3.760525461 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 53 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 49 | 3.760700040 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 3389 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 50 | 3.760801326 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 1723 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 51 | 3.760882831 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 22 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 52 | 3.760965158 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 1025 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 53 | 3.761077895 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 143 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 54 | 3.761240675 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 3306 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 55 | 3.806398518 | 192.168.1.1 | 10.0.3.15 | TCP | 60 | 53 → 54474 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 |
| 56 | 3.806486182 | 10.0.3.15 | 192.168.1.1 | TCP | 54 | 54474 → 53 [RST] Seq=1 Win=0 Len=0 |
| 57 | 3.809749765 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 113 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |
| 58 | 3.809932825 | 10.0.3.15 | 192.168.1.1 | TCP | 58 | 54474 → 23 [SYN] Seq=0 Win=1024 Len=0 MSS=1460 |

*Figure 12: Wireshark Capture of Nmap Scan Traffic*

This screenshot shows the network traffic captured in Wireshark during the Nmap scan. The packets highlight TCP SYN packets sent from the scanning host to the target system (192.168.1.1) as Nmap probes various ports to determine their status. The presence of multiple SYN packets and corresponding responses illustrates how port scanning works at the packet level, allowing the scanner to identify open, closed, or filtered ports on the target machine. This demonstrates how tools like Wireshark can be used to observe and analyze reconnaissance activities on a network.

# 4.Key Security Findings

| Protocol | Weakness | Risk level | Recommendation |
|---|---|---|---|
| HTTP | Plaintext Transmission | High | Use HTTPS/TLS |
| DNS | Unencrypted queries over UDP | Medium | Use DNS over HTTPS |
| TCP | SYN packets expose connection attempts | Low | Deploy firewall/IDS rules |
| ICMP | Reveals host availability | Low | Restrict ICMP on public-facing interfaces |

The main difference between HTTP and HTTPS is that in HTTPS the content would be shown in an encrypted format using TLS. Wireshark would show the TLS handshake process but the data would be in encrypted packets.

Plaintext communication is a security risk because the data is not encrypted, meaning anyone who intercepts the network traffic can easily read the information being transmitted, tools such as Wireshark can display the full contents of the communication, including webpage data, usernames, passwords, cookies, and other sensitive information.

Network traffic analysis also revealed that ICMP packets and TCP handshake packets are easily observable within the network capture, allowing analysts to identify communication patterns, host availability, and active services. The visibility of TCP SYN packets and connection attempts highlights how network reconnaissance and service discovery activities can be monitored using packet inspection tools. Additionally, the presence of retransmissions and sequence numbers demonstrates how TCP ensures reliability, but also exposes metadata such as source/destination endpoints, port numbers, and session timing, which could potentially be leveraged for traffic analysis or network mapping by an attacker.

# 5.Conclusion

This experiment demonstrated how Wireshark can be used to capture and analyze network traffic in real time. By examining different types of packets such as **DNS, ICMP, HTTP, and TCP**, it was possible to observe how network protocols operate and interact during normal communication. The analysis also showed how tools like packet filters and stream analysis help in understanding protocol behavior and troubleshooting network activity. Overall, Wireshark provides valuable insight into how data flows across a network and is an essential tool for network monitoring, troubleshooting, and cybersecurity analysis.